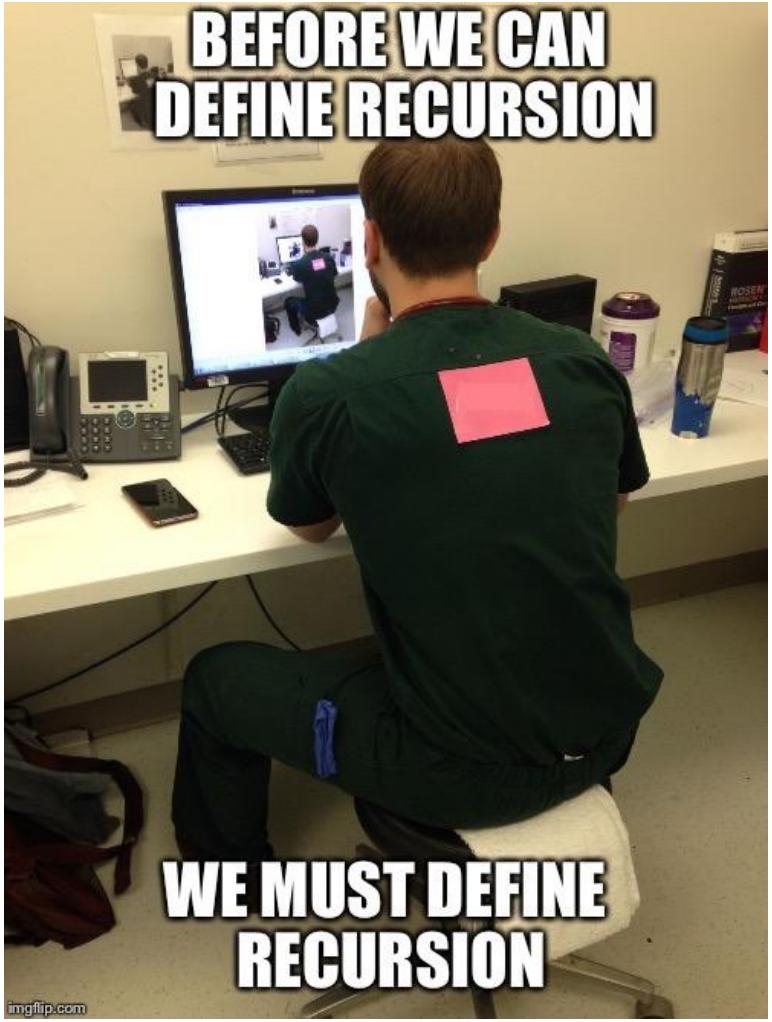
Introduction to Recursion


BEFORE WE CAN
DEFINE RECURSION
WE MUST DEFINE
RECURSION
imgflip.com

What is recursion?

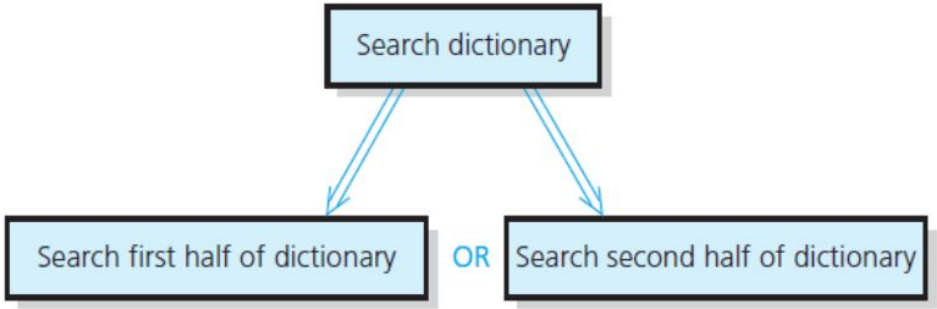
- A recursive function is a function that calls itself directly or indirectly.

- **Direct recursion**: when the function calls itself in the same function.

- **Indirect recursion**: when there is a chain of two or more function calls in which the original function is called again.

Why and when use recursion?

- Recursion breaks the original problem into smaller, identical sub-problems
- Divide and conquer approach.
- Recursive solutions tend to be inefficient, impractical. Typically, the iterative solution for the same problem is more efficient. Rule of thumb: avoid them!
- Can have infinite sequence of function calls (infinite recursion), similar to infinite loop, NOT GOOD.
- Many complex problems can lead to simple, elegant recursive solutions. Ex.:



Recursive solutions

- A recursive solution calls itself to solve a smaller sized problem than the original one.
- Must have at least one base case, which will avoid infinite recursive calls.
- The general case is the part of the solution that will call itself.
- Eventually the problem will be so small that it will be considered the base case, which in turn will return a concrete result.

Recursive solutions

Questions that help when constructing a recursion solution:

- Can the problem be solved by dividing it into smaller problems of the same type?
- How would each recursive call reduce the size of the problem?
- What is the smallest possible size of this problem? aka the base case.
- As the problem size diminishes, will it eventually reach the base case?

Recursive solutions: general format

- Each successive recursive call should bring you closer to a situation in which the answer is known.
- Each recursive algorithm must have at least one <u>base case</u> (stopping criteria) and one <u>general (recursive) case</u>.

Pseudo-code:

```
recursive_function () {
    if (some condition which the answer is known)
        solution statement          // base case
    else
        recursive_function() call   // general case
}
```

The Factorial of *n*

$$n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot (n-2) \cdot (n-1) \cdot n,$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120\,.$$

- An iterative solution

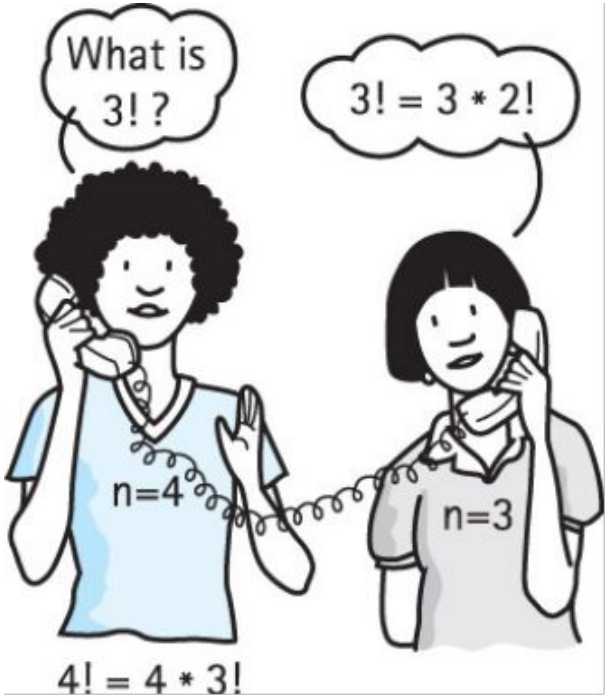
$$factorial(n) = n \times (n-1) \times (n-2) \times \cdots \times 1 \quad \text{for an integer } n > 0$$
$$factorial(0) = 1$$

- A recursive solution

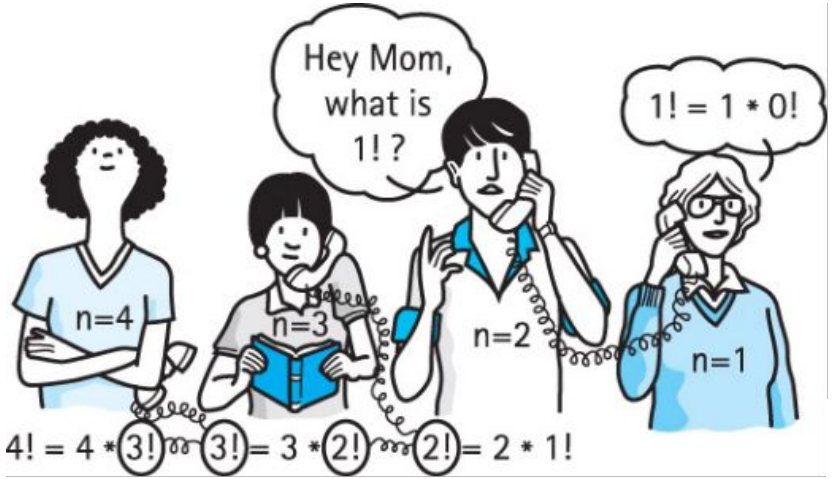
$$factorial(n) = \begin{cases} 1 & if\ n = 0 \\ n \times factorial(n-1) & if\ n > 0 \end{cases}$$

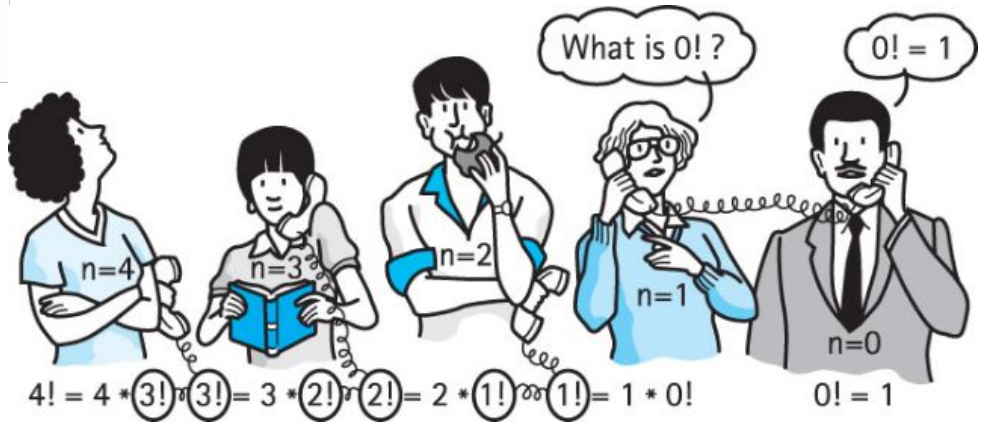
Note: avoid using a recursion if there is a simple efficient iterative solution.

Recursive query

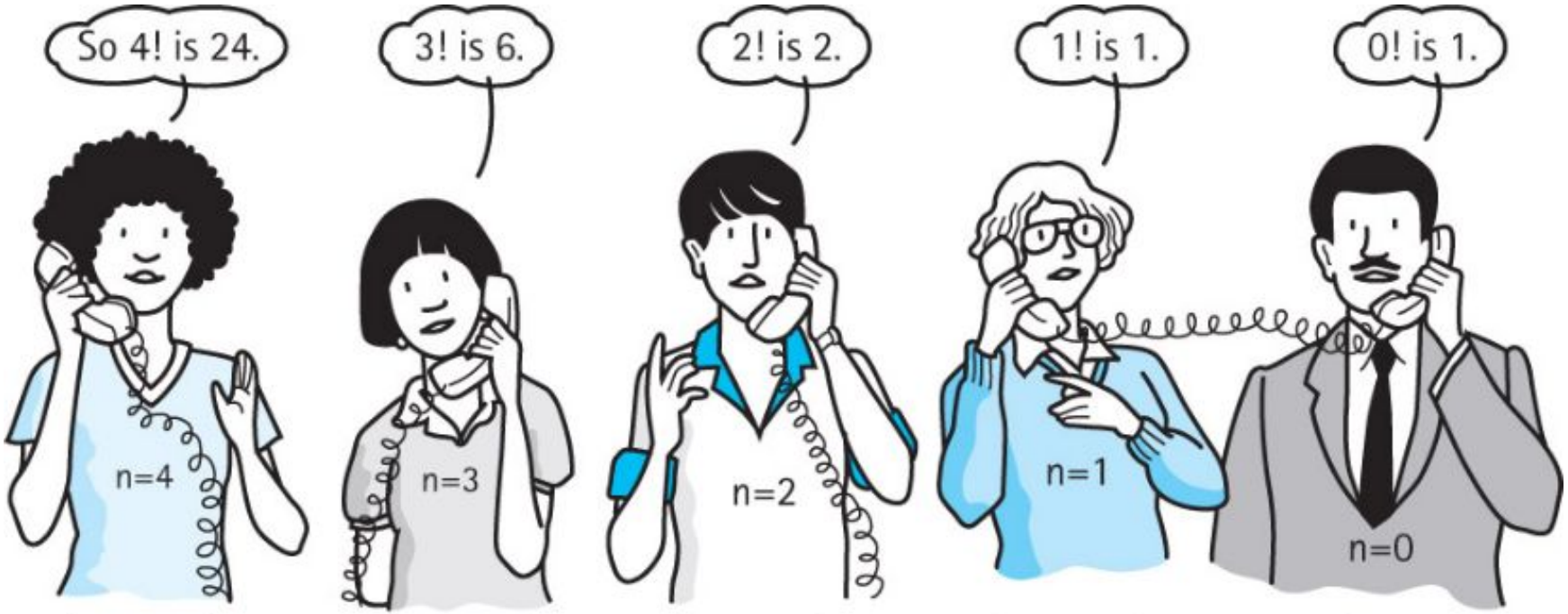


Recursive query





Recursive solution



$4! = 4 * 3! =$
$4 * 6 = 24$

$3! = 3 * 2! =$
$3 * 2 = 6$

$2! = 2 * 1! =$
$2 * 1 = 2$

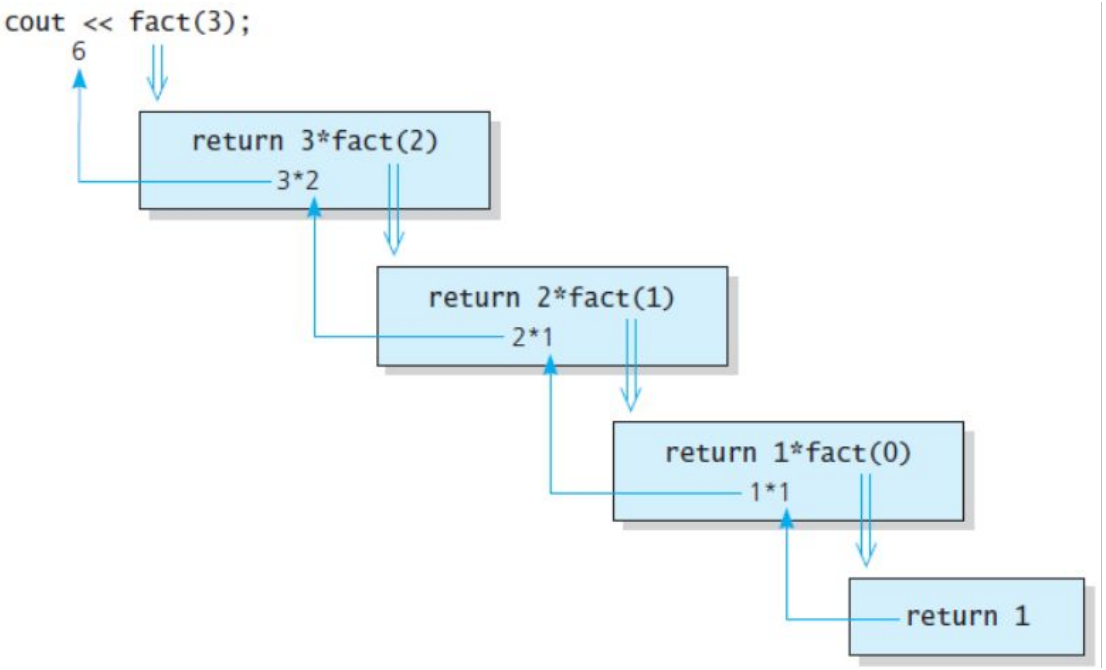
$1! = 1 * 0! =$
$1 * 1 = 1$

$0! = 1$

Recursive valued function: factorial

```c
/**
 * Computes the factorial of the non-negative integer 'n'
 */
int factorial (int n)
{
    if (n == 0) {
        return 1;
    }
    else {
        return n * factorial(n - 1);
    }
}
```

Recursive call chain

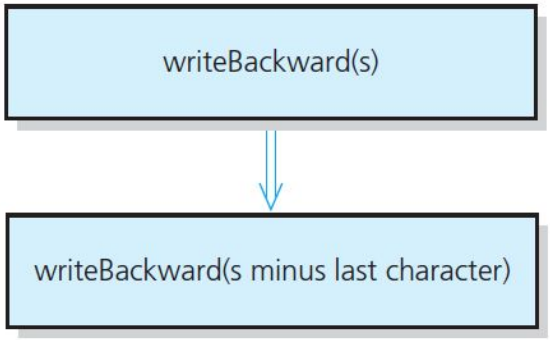



```
factorial(4);
4 * factorial(3);
4 * (3 * factorial(2));
4 * (3 * (2 * factorial(1)));
4 * (3 * (2 * (1 * factorial(0))));
4 * (3 * (2 * (1 * 1)));
24;
```

Note: all the unsolved calls will remain in memory until the base case is reached!

Recursive void function: write a string backwards

```cpp
/**
 * Prints a string to the terminal backwards.
 * Ex.: backwards("abc") -> "cba"
 */
void backwards(std::string s)
{
    int lengthOfString = s.size();
    // General case: string has at least 1 character
    if (lengthOfString > 0) {
        std::cout << s.substr(lengthOfString - 1, 1);
        backwards(s.substr(0, lengthOfString - 1));
    }
    // Base case: no characters left, do nothing
    else {
        std::cout << "\n";
    }
}
```



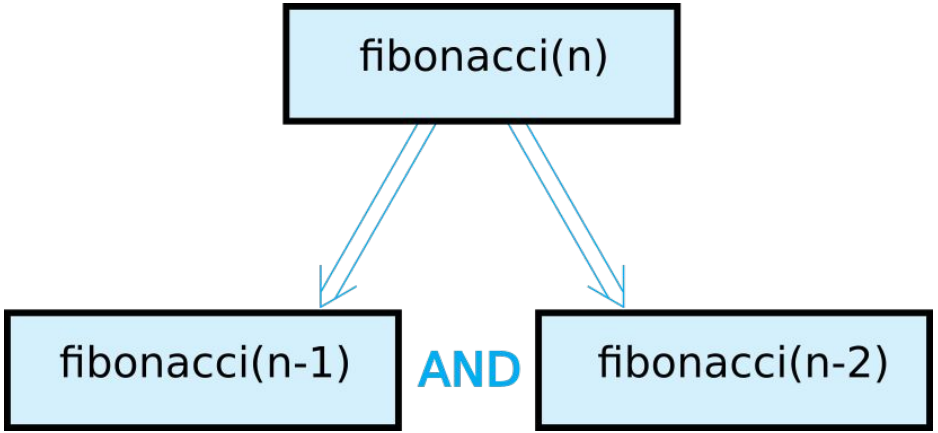

```cpp
backwards("abcd"); // length = 4: print last char "d", call backwards("abc")
backwards("abc");  // length = 3: print last char "c", call backwards("ab")
backwards("ab");   // length = 2: print last char "b", call backwards("a")
backwards("a");    // length = 1: print last char "a", call backwards("")
backwards("");     // length = 0: skip general case, reach base case, print \n
```

Note: Even though there is no return statement, all calls will remain in memory until base case is reached.

The Fibonacci Sequence

$$F_n = F_{n-1} + F_{n-2},$$

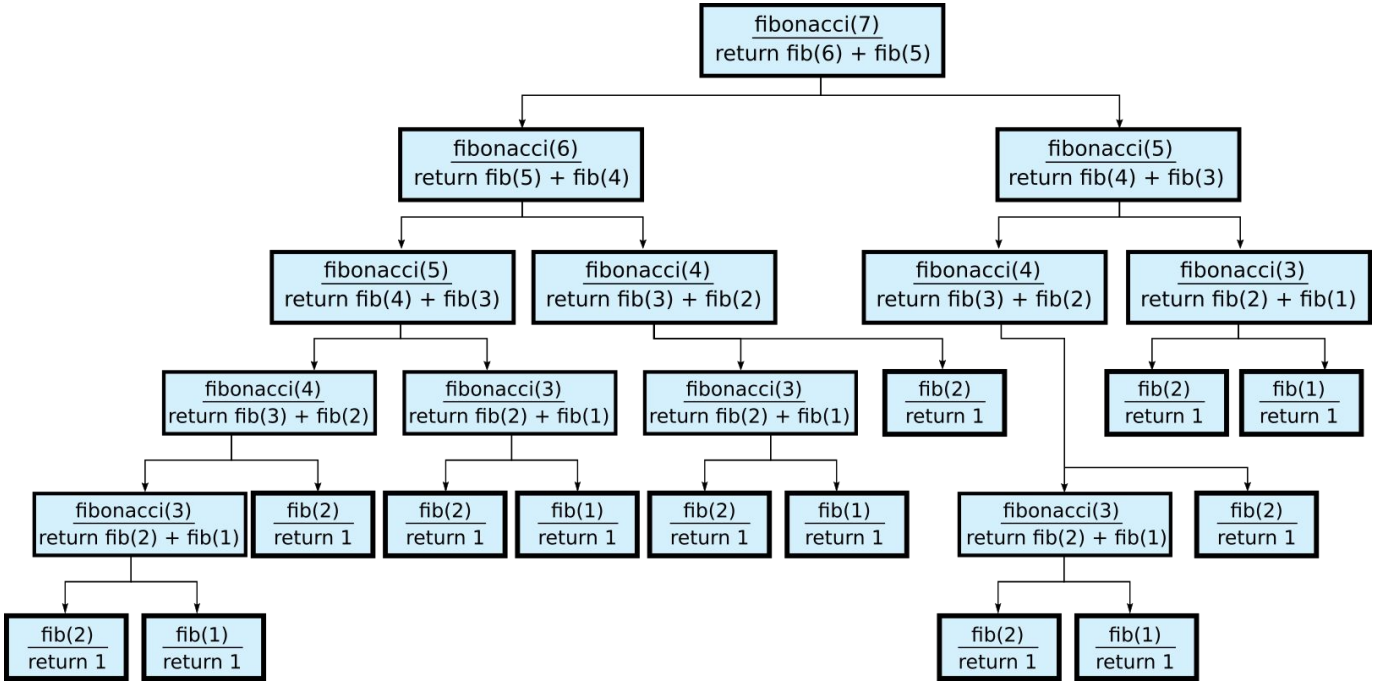
$$(0,)\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ 89,\ 144,\ \ldots$$



The Fibonacci Sequence

```c
/**
 * Finds the n-th number in the Fibonacci sequence
 */
int fibonacci(int n)
{
    if (n <= 2) {   // Base case
        return 1;
    }
    else {          // General case: results in two more calls
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

The Fibonacci Sequence: generated recursive calls



The n-choose-k problem

$$\binom{n}{k} = \frac{n(n-1)\cdots(n-k+1)}{k(k-1)\cdots 1}, \qquad \binom{n}{k} = \frac{n!}{k!(n-k)!},$$

- Given a set of *n* things, how many ways can you choose *k* of them, given that *n* >= *k*?
- "A band wants to tour *k* out of *n* cities."
  - The order is not an issue
- Let *g(n, k)* be the number of groups of *k* cities chosen from *n*, then:

$$g(n,k) = g(n-1,k-1) + g(n-1,k)$$

- Base cases:

$$g(k,k) = 1$$
$$g(n,0) = 1$$

Combinations

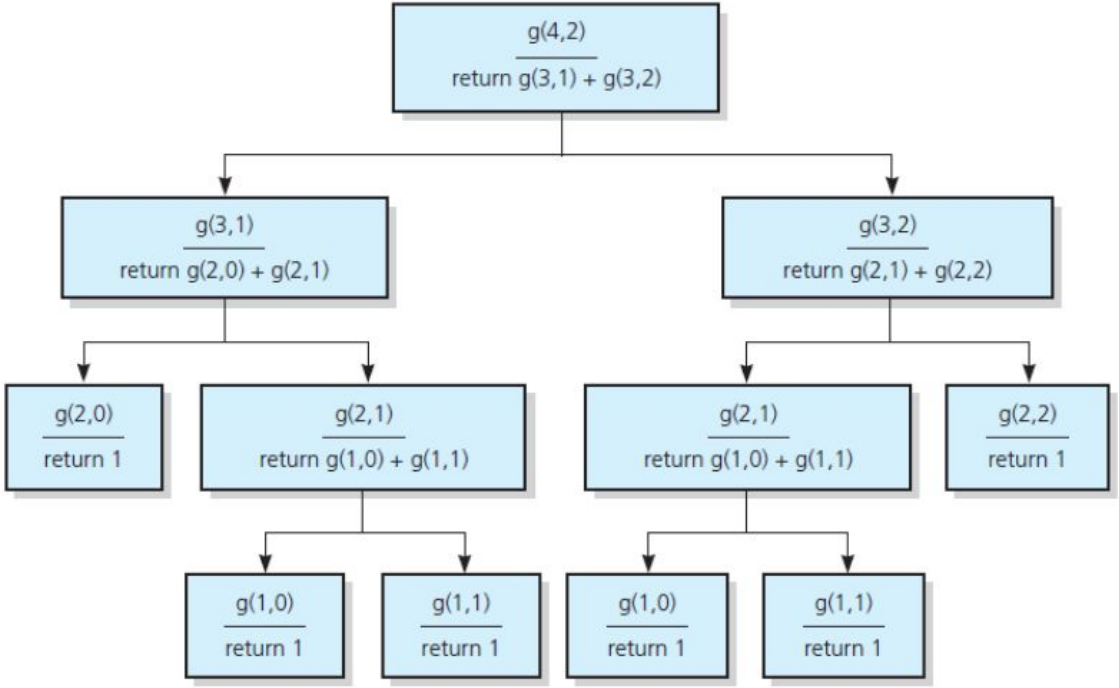
How many combinations of a certain size can be made out of a total group of elements?

$$g(n,k) = \begin{cases} 1 & \text{if } k = 0 \\ 1 & \text{if } k = n \\ 0 & \text{if } k > n \\ g(n-1,k-1) + g(n-1,k) & \text{if } 0 < k < n \end{cases}$$

n-choose-k solution

```
/**
 * Answers the question: How many combinations of size 'k'
 * can be made from a group (set) of elements of size 'n'?
 */
int fromNchooseK(int n, int k)
{
    if (k == 0 || k == n) {
        return 1;
    }
    else if (k > n) {
        return 0;
    }
    else {
        return fromNchooseK(n - 1, k - 1) + fromNchooseK(n - 1, k);
    }
}
```

Calls generated (*n*=4, *k*=2)



Tail Recursion

- When a function can pre-compute the result and pass it along to the subsequent recursive calls. The next recursive call simply updates the pre-computed value.
- Tail recursive solutions may need less memory, depending on the cimpiler.
- Tail recursion can be replaced by iteration, effectivelly removing recursion from the solution.

Cumulative sum

- The sum of the first *N* integers: $f(n) = n + (n-1) + (n-2) + \ldots + 1,\ n >= 1$
- Typical recursive solution:

```c
/**
 * Computes the sum of the first n-th integers
 * using conventional recursion.
 */
int recsum(int n)
{
    if (n == 1) {
        return 1;
    }
    else {
        return n + recsum(n - 1);
    }
}
```

Cumulative sum: tail-recursive solution

```c
/**
 * Computes the sum of the first n-th integers
 * using tail recursion.
 */
int tailrecsum(int n, int partial_result)
{
    if (n == 0) {
        return partial_result;
    }
    else {
        return tailrecsum(n - 1, partial_result + n);
    }
}
```

Call comparison

```
recsum(5);
5 + recsum(4);
5 + (4 + recsum(3));
5 + (4 + (3 + recsum(2)));
5 + (4 + (3 + (2 + recsum(1))));
5 + (4 + (3 + (2 + 1)));
15;
```

```
tailrecsum(5, 0);
tailrecsum(4, 5);
tailrecsum(3, 9);
tailrecsum(2, 12);
tailrecsum(1, 14);
tailrecsum(0, 15);
15;
```

In the conventional way, the program has to wait for all the recursive calls to finish and then actually executing the sum. The tail-recursive updates the result at each recursive call.

Iterative solution

```c
/**
 * Computes the sum of the first n-th integers
 * using iterations.
 */
int itersum(int n)
{
    int result = 0;
    for (int i = 0; i <= n; i = i + 1) {
        result = result + i;
    }
    return result;
}
```

Recursion and Efficiency

- Factors that contribute to inefficiency:
  - Overhead associated with function calls.
  - Repeated recursive calls: some algorithms execute the same task multiple times
    - e.g. the Fibonacci sequence
- Keep in mind:
  - Recursion can clarify complex solutions. However ...
  - Clear, efficient, iterative solutions are better.
- Why do we need to know recursion at all?
  - Interviews
  - Sometimes quicker to find a first solution

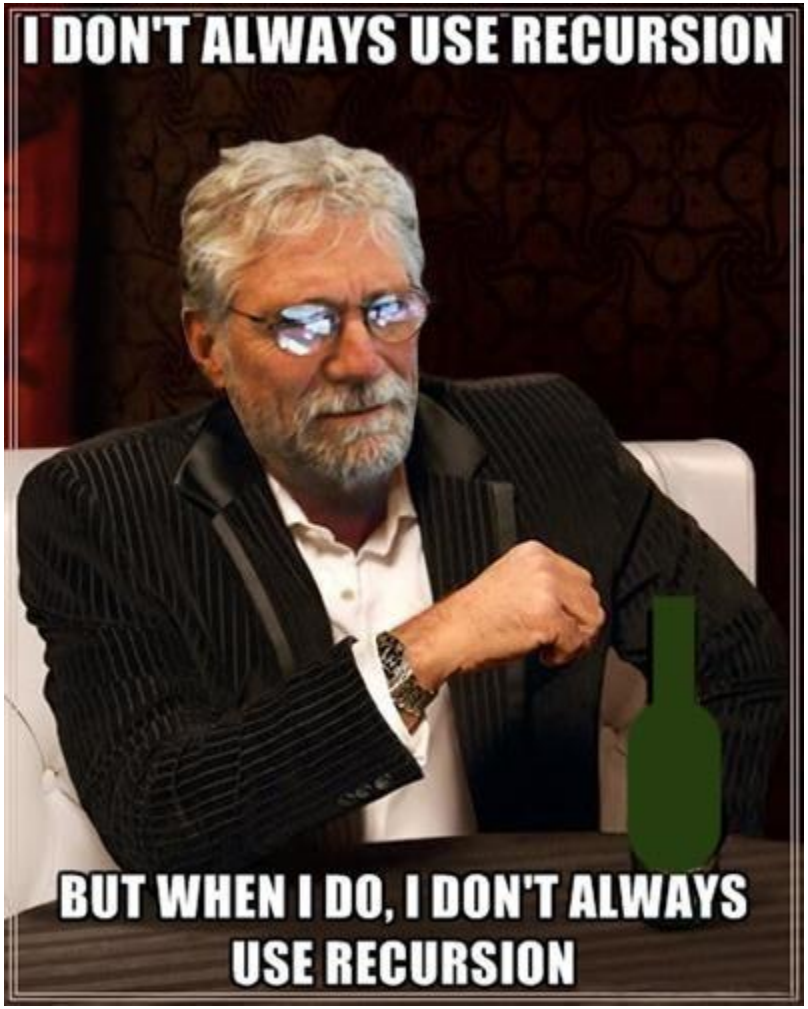
When to use recursive solution

- When the depth of the recursive calls is relatively "**shallow**" compared to the size of the problem.
- When the recursive solution does **less** work than its iterative counterpart.
- When the recursive version is **much** shorter and simpler than the non-recursive solution.

SHALLOW DEPTH

EFFICIENCY

CLARITY

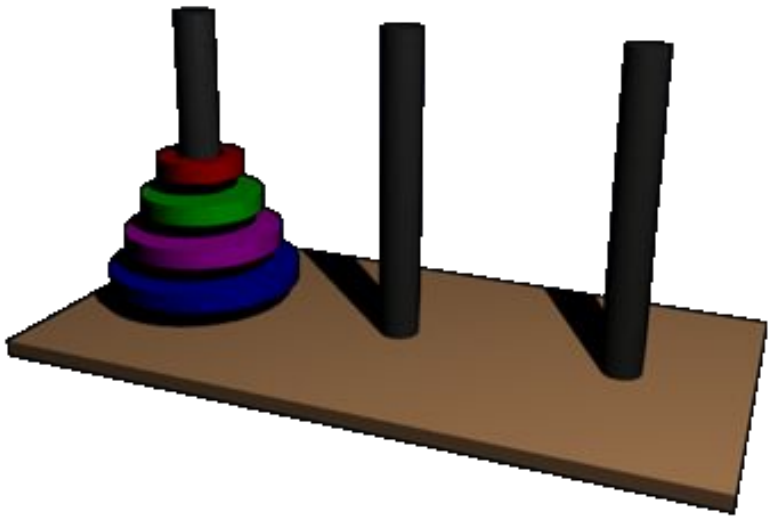

I DON'T ALWAYS USE RECURSION
BUT WHEN I DO, I DON'T ALWAYS
USE RECURSION

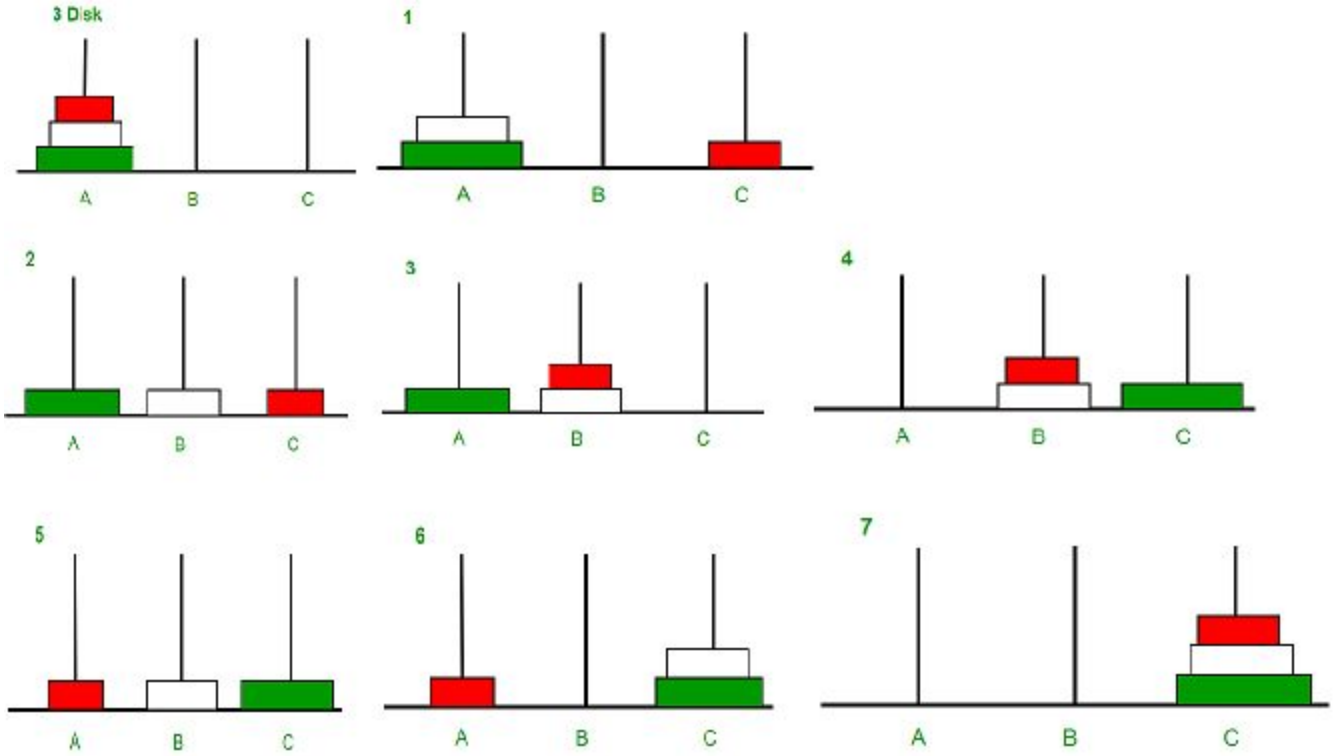
Recursion is not difficult, needs practice!

Some more examples:

- Tower of Hanoi
- Binary Search

Towers of Hanoi




3 Disk
A
B
C
1
A
B
C
2
A
B
C
3
A
B
C
4
A
B
C
5
A
B
C
6
A
B
C
7
A
B
C

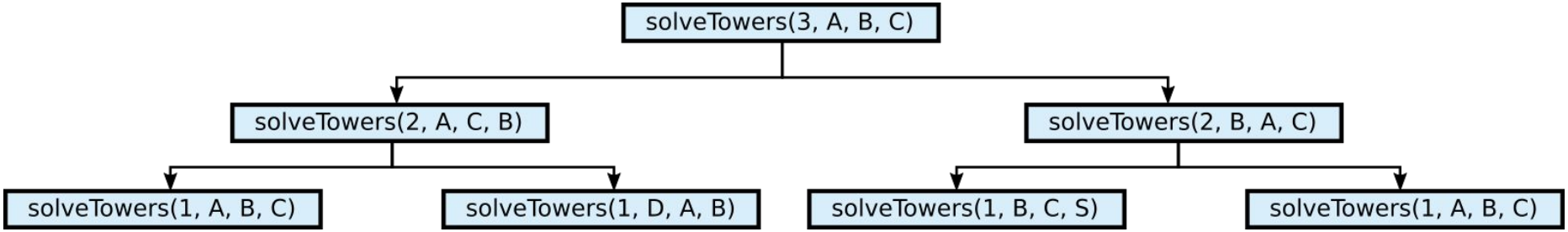
Towers of Hanoi

- Problem: starting with *n* disks on pole **A** and zero on poles **B** and **C**. Move all disks to pole **C** without having a larger disk resting on top of a smaller disk. Solve function *solveTowers(n, A, B, C)*.
- Solution:
  - With all disks on **A**, solve *solveTowers(n-1, A, C, B)*.
  - With largest disk on **A**, and all others on **C**, *solveTowers(n-1, A, B, C)*.
  - With largest disk on **B**, and all others on **C**, *solveTowers(n-1, C, B, A)*.

Towers of Hanoi: solution

```
/**
 * Prints all the movements to complete the Towers of Hanoi game
 * with an arbitrary number of disks.
 */
void solveTowers (int n, char from, char to, char aux)
{
    if (n == 1)
    {
        std::cout << "Move #1: " << from << " -> " << to << "\n";
        return;
    }
    solveTowers(n - 1, from, aux, to);
    std::cout << "Move #" << n << ": " << from << " -> " << to << "\n";
    solveTowers(n - 1, aux, to, from);
}
```

Towers of Hanoi: recursive calls



Minimal number of moves: $2^n - 1$

Example iterative solution: https://www.geeksforgeeks.org/iterative-tower-of-hanoi/

Binary Search

- Given a **sorted array**, find the position of a given *target value*.
- Example array: ascending prime numbers

Index	0	1	2	3	4	5	6	7
Value	2	3	5	7	11	13	17	23

- Solution:
  - Find the middle point
    - If *middle* point is *target* value, return
    - If *middle* point is greater than *target*, repeat procedure with left half of list
    - If *middle* point is lower than *target*, repeat procedure with right half of list
    - Return -1 if *target* not found

Binary Search

```cpp
/**
 * Tries to find an element's index position on a given sorted array
 * returns the index position or '-1' if value not found
 */
int binarySearch (int array[], int left, int right, int target)
{
    if (right >= left) {
        int middle = left + (right - left) / 2;
        if (array[middle] == target) {
            return middle;
        }
        if (array[middle] > target) {
            return binarySearch(array, left, middle - 1, target);
        }
        return binarySearch(array, middle + 1, right, target);
    }
    return -1;
}
```

```cpp
int main ()
{
    //              0  1  2  3   4
    int array[] = { 2, 3, 4, 10, 40 };
    int target = 10;
    int length = 5; // sizeof(array) / sizeof(array[0]);
    int result = binarySearch(array, 0, length - 1, target);
    if (result == -1) {
        std::cout << "Target value not found\n";
    }
    else {
        std::cout << "Target value at index " << result << "\n";
    }
}
```

Summarizing

- The challenge in using recursion to solve a problem relies on identifying that a problem *CAN* be solved by recursion.
- Determining the base case and make sure it will eventually be triggered, otherwise you'll end up with an infinite recursion call.
- Important concept to understand, even if you don't use it in your future job.
- Don't neglect recursion, sometimes an interview question can be solved by using it (and with fewer lines of code).

Questions?